Chapter 5 — Objects, Math, Date and JSON

Almost everything interesting in a JavaScript program is an **object**: a user profile, an HTTP response (the reply a web server sends back to a request — see System Design Fundamentals), a configuration file, a DOM element (one piece of a web page, like a button, as the browser represents it to JavaScript), even an array or a function. This chapter teaches the everyday object toolkit — creating objects, reading and changing their properties, copying and merging them, locking them down — and then three built-in objects you will use constantly: `Math` for numbers, `Date` for time, and `JSON` for sending data between programs.

What you'll learn

1. What an object is, object literals, and how keys and values are stored.
2. Dot vs bracket access, computed keys, shorthand properties, adding/updating/deleting properties, checking whether a key exists, key order.
3. Methods and `this` inside a method (the short version — the full rules are in Chapter 9).
4. Spread and rest with objects, shallow vs deep copies, and destructuring.
5. The `Object.*` static methods: `keys`, `values`, `entries`, `fromEntries`, `assign`, `create`, `is`, and the reflection/locking family.
6. Property descriptors, `freeze` / `seal` / `preventExtensions`, getters and setters.
7. Optional chaining `?.` for safely reading deep data.
8. The built-in ("native") objects at a glance.
9. `Math`: constants, rounding, min/max, powers, trigonometry, random numbers.
10. `Date`: creating, reading, changing, comparing and formatting dates — plus the state of **Temporal**, its modern replacement.
11. `JSON.stringify` and `JSON.parse`, including replacers, revivers and what JSON cannot represent.

Before you start. You should know variables, primitive types and operators (Chapter 2) and functions and arrow functions (Chapter 4). Arrays get their own chapter (Chapter 7), and `Map` / `Set` — which are often better than plain objects for lookup tables — are in Chapter 8.

All runnable code is in `code/`. Every file is an ES module (`.mjs` — a file that uses the modern `import`/`export` system, Chapter 10), which means it runs in **strict mode** (a stricter variant of JavaScript where some silent failures become errors). Outputs shown are real, produced with Node 24.

5.1 What an object is

What it is

An **object** is a collection of **properties**. Each property is a **key** (a name, normally a string) paired with a **value** (any JavaScript value — a number, a string, another object, a function). A property whose value is a function is called a **method**.

Think of an object as a labelled filing box: every drawer has a label (the key) and holds something (the value). A real car is a good mental model: it has *properties* such as colour, brand and model, and it can *do* things such as drive or brake — those actions are its methods.

Compare that with a **primitive** (string, number, boolean, `null`, `undefined`, bigint, symbol), which is a single, unchangeable value. An object groups many related values — and the behaviour that works on them — under one name.

Why it matters

Real data is structured. A user is not "a string"; it is a name, an email, a list of roles, a signup date. Objects let you keep those pieces together, pass them around as one thing, and attach behaviour to them. JSON (section 5.12), the format of nearly every web API (a service other programs call over the internet to fetch or send data), is basically a text version of JavaScript objects and arrays.

How it works — creating objects with a literal

The simplest and by far the most common way to make an object is an **object literal**: curly braces containing `key: value` pairs separated by commas.

```
const car = {
  color: 'red',
  brand: 'Toyota',
  model: 'Corolla',
  drive() {                       // a method
    console.log('The car is moving');
  },
};
```

- `{ ... }` creates a new object each time this line runs.
- `color`, `brand`, `model` are properties holding strings.
- `drive` is a method. `drive() { ... }` is **method shorthand** (ES2015) for `drive: function () { ... }`.
- A trailing comma after the last property is allowed and makes diffs (the line-by-line "what changed" view in version control) cleaner.

Other ways to create objects exist — `new Object()` (never needed), `Object.create(proto)` (section 5.6), constructor functions and classes (Chapter 9) — but literals cover 90% of everyday code.

5.2 Reading, adding, updating and deleting properties

Dot notation vs bracket notation

There are two ways to reach a property:

	Dot notation `obj.key`	**Bracket notation** `obj['key']` / `obj[expr]`
Key must be a valid identifier?	Yes (letters, digits, `_`, `$`; cannot start with a digit; no spaces or dashes)	No — any string (or symbol, a unique key type from Chapter 8) works
Key known while writing the code?	Yes — the name is typed literally	Not necessary — the key can come from a variable or expression
Readability	Cleaner; the default choice	A little noisier
Typical use	`user.name`, `config.port`	`headers['content-type']`, `row[columnName]`, `scores[2024]`

Rule of thumb: **use dot notation unless you can't** — the key has special characters, or it is only known at run time (while the program is running, not when you write it).

Computed keys and shorthand properties

Two ES2015 features make literals more flexible:

- **Computed keys** — write `[expression]: value` and the *result* of the expression becomes the key.
- **Shorthand properties** — `{ name, age }` means `{ name: name, age: age }` when variables with those names exist.

`code/01_object_basics.mjs` (first part):

```
const field = 'price';

const laptop = {
  brand: 'Lenovo',            // plain property
  'model name': 'ThinkPad',   // key that is not a valid identifier -> must be quoted
  [field]: 1299,              // computed key: the value of `field` becomes the key
  [`${field}WithTax`]: 1299 * 1.2,
  open() {                    // method shorthand
    return `${this.brand} ${this['model name']} is open`;
  },
};

console.log(laptop.brand);                // known, valid identifier
console.log(laptop['model name']);        // spaces -> brackets required
const key = 'price';
console.log(laptop[key], laptop.priceWithTax);
console.log(laptop.color);                // missing property -> undefined, not an error

const name = 'Ada', age = 36;
const user = { name, age };
console.log(user);
```

Output:

```
Lenovo
ThinkPad
1299 1558.8
undefined
{ name: 'Ada', age: 36 }
```

Reading a property that does not exist gives `undefined` — no error. Reading a property *of* `undefined` (for example `laptop.color.length`) **is** an error (`TypeError`); section 5.8 shows how `?.` avoids it.

Adding, updating and deleting

Objects are **mutable**: you can change them after creation. Assigning to a key that exists updates it; assigning to a new key adds it. `delete obj.key` removes a property entirely.

```
laptop.brand = 'Dell';          // update (dot)
laptop['ram'] = '16GB';         // add (bracket)
delete laptop.priceWithTax;     // remove
console.log(laptop.open(), '|', Object.keys(laptop).join(','));
```

Output:

```
Dell ThinkPad is open | brand,model name,price,open,ram
```

Does the object have this key?

```
console.log('ram' in laptop, Object.hasOwn(laptop, 'ram'), laptop.ram !== undefined);
const withUndefined = { x: undefined };
console.log('x' in withUndefined, withUndefined.x !== undefined);
```

Output:

```
true true true
true false
```

Check	Looks at inherited (prototype) keys?	Works when the value is `undefined`?
`'key' in obj`	Yes (so `'toString' in {}` is `true`)	Yes
`Object.hasOwn(obj, 'key')` (ES2022)	No — own properties only	Yes
`obj.hasOwnProperty('key')`	No	Yes, but fails on objects without a prototype; prefer `Object.hasOwn`
`obj.key !== undefined`	Yes	**No** — a key explicitly set to `undefined` looks missing

(*Inherited* keys come from the object's **prototype** — a fallback object JavaScript checks when a key isn't found on the object itself; section 5.6.)

Key order

People often say "objects have no order". That was never quite true and has been fully specified since ES2020 for the common operations (`Object.keys`, `for...in`, `JSON.stringify`, …). The order of own keys is:

1. **Integer-like keys** (`'0'`, `'1'`, `'42'`) in ascending numeric order;
2. then other **string keys** in the order they were added;
3. then **symbol keys** in the order they were added (only reported by functions that include symbols).

```
const ordered = { b: 1, 2: 'two', a: 2, 1: 'one', [Symbol('s')]: 3 };
console.log(Object.keys(ordered));
console.log(Reflect.ownKeys(ordered));
```

Output:

```
[ '1', '2', 'b', 'a' ]
[ '1', '2', 'b', 'a', Symbol(s) ]
```

So the order is deterministic (always the same for the same input), but it is *not* pure insertion order when keys look like numbers. If you need a keyed collection where insertion order always wins (or keys that are not strings), use a `Map` (Chapter 8).

Objects are references

A variable holding an object holds a **reference** — an arrow pointing at the object — not the object itself. Copying the variable copies the arrow.

```
const a = { n: 1 };
const b = a;           // same object, second name
b.n = 99;
console.log(a.n, a === b, { n: 1 } === { n: 1 });

const cfg = { debug: false };
cfg.debug = true;      // allowed
console.log(cfg);
try { cfg = {}; } catch (e) { console.log(e.constructor.name + ':', e.message); }
```

Output:

```
99 true false
{ debug: true }
TypeError: Assignment to constant variable.
```

- `===` on objects asks "same object?", not "same contents?". Two separate literals with equal contents are never `===`.
- `const` freezes the **binding** (the variable cannot point elsewhere), not the object. To stop mutation, use `Object.freeze` (section 5.6).

Common mistakes

- Writing `obj.key` when the key is in a variable: `obj.field` looks for a property literally named `"field"`. Use `obj[field]`.
- Using a number-like string key and expecting insertion order.
- Believing `const` makes an object immutable.
- Comparing objects with `===` to test "equal contents" — compare the fields, or serialize both deliberately (convert each to a string such as JSON and compare the strings).

Self-check

1. How do you read a property named `first-name`? → `obj['first-name']` — the dash makes it an invalid identifier.
2. `const k = 'x'; ({ [k]: 1 })` produces what? → `{ x: 1 }`.
3. `'toString' in {}` vs `Object.hasOwn({}, 'toString')`? → `true` (inherited) vs `false` (not own).
4. What does `Object.keys({ z: 1, 10: 1, 2: 1 })` return? → `['2', '10', 'z']`.

5.3 Methods and `this`

What it is

A **method** is a function stored as a property. When you call it as `obj.method()`, JavaScript sets the special keyword `this` inside the function to `obj` — the object to the left of the dot. That is how one method can read the object's other properties.

```
const person = {
  firstName: 'Grace',
  lastName: 'Hopper',
  greet() {
    console.log(`Hi, I'm ${this.firstName} ${this.lastName}`);
  },
};
person.greet();   // Hi, I'm Grace Hopper
```

Methods can be called with brackets too: `person['greet']()`.

Common mistakes

- **Arrow functions as methods.** Arrow functions do not get their own `this`; they borrow it from the surrounding code. An arrow method on an object literal therefore does *not* see the object:

```
const counter = {
  count: 0,
  incArrow: () => typeof this,          // `this` is the module's this (undefined)
  inc() { this.count++; return this.count; },
};
console.log(counter.incArrow(), counter.inc());
```

Output (from `code/05_getters_setters_optional.mjs`):

```
undefined 1
```

- **Detaching a method.** `const f = person.greet; f();` loses `this` (it becomes `undefined` in strict mode). The full set of rules — default, implicit, explicit (`call`/`apply`/`bind`) and `new` binding — is in Chapter 9, section 9.1.

Returning `this` for chaining

A method that returns `this` lets calls be chained, the style used by jQuery (a classic library for manipulating web pages), query builders (libraries that build database queries step by step) and many test libraries:

```
deposit(amount) { this._balance += amount; return this; }
// account.deposit(10).deposit(5)
```

5.4 Spread, rest and copying objects

What it is

- **Spread** `{ ...source }` (ES2018) copies the *own* (not inherited), *enumerable* (visible to loops and listing methods) properties of `source` into a new object literal.
- **Rest** in destructuring `const { a, ...others } = obj` collects "everything I didn't name" into a new object.

Both use the same three dots; spread *expands* an object into a literal, rest *gathers* leftovers into a variable.

How it works

1. Properties are copied left to right.
2. If the same key appears twice, the **later one wins** — this is how you apply overrides on top of defaults.
3. Only the top level is copied. Nested objects are shared (a **shallow copy**).

From `code/02_object_static_methods.mjs`:

```
const defaults = { theme: 'light', lang: 'en', fontSize: 14 };
const prefs = { theme: 'dark' };
const merged = Object.assign({}, defaults, prefs); // later sources win
console.log(merged);

console.log({ ...defaults, ...prefs, fontSize: 16 });

const { password, ...safeUser } = { id: 7, name: 'Lin', password: 'x' };
console.log(safeUser);

const original = { name: 'Kit', address: { city: 'Pune' } };
const shallow = { ...original };
const deep = structuredClone(original);
shallow.address.city = 'Delhi';
console.log(original.address.city, deep.address.city);
```

Output:

```
{ theme: 'dark', lang: 'en', fontSize: 14 }
{ theme: 'dark', lang: 'en', fontSize: 16 }
{ id: 7, name: 'Lin' }
Delhi Pune
```

The last line is the classic bug: changing `shallow.address.city` also changed `original`, because both point to the *same* inner address object. **`structuredClone(value)`** (a global in browsers and Node 17+) makes a true **deep copy** — it copies nested objects, arrays, `Date`, `Map`, `Set`, typed arrays and even circular references (an object that points back to itself, directly or through others). It cannot copy functions, DOM nodes or class identity (instances — objects created from a class — come back as plain objects).

Trade-offs: ways to copy or merge

(A *getter* is a property that runs a function when read; a *setter* runs one when written — section 5.7.)

Technique	Depth	Mutates target?	Copies getters as getters?	Notes
`{ ...a, ...b }`	Shallow	No (new object)	No — getters are *called*, values copied	Most common; readable
`Object.assign(target, a, b)`	Shallow	**Yes** — writes into `target`	No — getters called, setters on target triggered	Use `Object.assign({}, ...)` for a fresh object
`structuredClone(obj)`	Deep	No	No	Handles Date/Map/Set/cycles; throws on functions
`JSON.parse(JSON.stringify(obj))`	Deep	No	No	Lossy: Dates → strings, drops `undefined`/functions, fails on cycles and BigInt (section 5.12)

Real-world examples

- Redux reducers (functions that compute an app's next state in the Redux library) and React state updates (changing the data a React component displays): `return { ...state, loading: false }` — create a new object instead of mutating.
- Config loaders: `{ ...defaultConfig, ...fileConfig, ...envConfig }` — later layers override earlier ones.
- Removing sensitive fields before logging or sending: `const { password, ...safe } = user`.

In the interview

> **30-second answer:** "Spread copies an object's own enumerable properties into a new literal; later keys override earlier ones. It's shallow — nested objects are shared. For a deep copy I use `structuredClone`, which handles Dates, Maps, Sets and cycles; the JSON round-trip is lossy."

- *Is `Object.assign` the same as spread?* Nearly; but `assign` mutates its first argument and triggers setters on it, while spread always creates a new object.
- *How would you deep-merge two configs?* Recurse: for each key, if both values are plain objects, merge them recursively; otherwise take the right-hand value. Or use a library such as lodash `merge` (lodash is a popular collection of utility functions).

Self-check

1. `{ ...{ a: 1 }, a: 2 }` → `{ a: 2 }`.
2. After `const c = { ...o }`, does `c.list.push(1)` change `o.list`? → Yes; shallow copy.
3. What does `structuredClone` do with a method? → Throws a `DataCloneError`.

5.5 Destructuring objects

What it is

Destructuring (ES2015) unpacks properties into variables in one statement: `const { a, b } = obj` is short for `const a = obj.a, b = obj.b`.

How it works

From `code/03_destructuring.mjs`:

```
const order = {
  id: 'A-17',
  customer: { name: 'Mira', address: { city: 'Lyon' } },
  items: 3,
};

const { id, items } = order;                      // basic
const { id: orderId } = order;                    // rename: take `id`, call it orderId
const { status = 'pending' } = order;             // default when the value is undefined
const { customer: { name, address: { city } } } = order; // nested
console.log(id, items, orderId, status, name, city);

const { a = 1, b = 2 } = { a: undefined, b: null };
console.log(a, b);

function createUser({ name, role = 'viewer', active = true } = {}) {
  return `${name ?? 'anonymous'} / ${role} / ${active}`;
}
console.log(createUser({ name: 'Jo', role: 'admin' }));
console.log(createUser());

let x, y;
({ x, y } = { x: 10, y: 20 });
console.log(x, y);

try { const { q } = null; } catch (e) { console.log(e.constructor.name); }
```

Output:

```
A-17 3 A-17 pending Mira Lyon
1 null
Jo / admin / true
anonymous / viewer / true
10 20
TypeError
```

Key points:

- `{ id: orderId }` reads as "take property `id`, store it in `orderId`" — the name on the **right** of the colon is the new variable.
- Defaults (`= 'pending'`) apply only when the value is `undefined` — **not** when it is `null`.
- In the nested pattern `customer: { name }`, `customer` is only a path; no `customer` variable is created.
- **Parameter destructuring** gives JavaScript "named arguments": callers pass an object, order does not matter, and each option has its own default. The trailing `= {}` makes calling with no argument safe.
- Assigning to *existing* variables needs parentheses, otherwise `{` starts a block.
- Destructuring `null` or `undefined` throws a `TypeError`.

Array destructuring (`const [first, second] = arr`) is covered in Chapter 7.

Self-check

1. `const { a: { b } } = { a: { b: 5 } }` — which variables exist? → Only `b` (value 5).
2. `const { n = 10 } = { n: 0 }` → `n` is `0`; defaults only replace `undefined`.
3. Why write `function f({ x } = {})`? → So `f()` with no argument does not throw.

5.6 `Object.*` static methods, descriptors and locking

What it is

The global `Object` holds **static methods** — helpers you call as `Object.method(obj)`, not on the object itself. They fall into four groups:

Group	Methods	What they do
Listing	`Object.keys(o)`, `Object.values(o)`, `Object.entries(o)`, `Object.fromEntries(pairs)`	Own enumerable string-keyed names / values / `[key, value]` pairs; `fromEntries` rebuilds an object from pairs
Copying & creating	`Object.assign(target, ...sources)`, `Object.create(proto, descriptors?)`	Copy properties into a target; make a new object with a chosen prototype
Reflection (code inspecting the structure of objects at run time)	`Object.getOwnPropertyNames(o)` (includes non-enumerable string keys), `Object.getOwnPropertySymbols(o)`, `Object.getOwnPropertyDescriptor(o, k)`, `Object.getOwnPropertyDescriptors(o)`, `Object.getPrototypeOf(o)`, `Object.setPrototypeOf(o, p)`, `Object.hasOwn(o, k)`, `Object.is(a, b)`	Inspect keys, flags and the prototype; compare values
Defining & locking	`Object.defineProperty(o, k, desc)`, `Object.defineProperties(o, descs)`, `Object.preventExtensions`, `Object.seal`, `Object.freeze`, and the tests `isExtensible`, `isSealed`, `isFrozen`	Control exactly how a property behaves; stop an object from changing

Listing: keys, values, entries, fromEntries

From `code/02_object_static_methods.mjs`:

```
const stock = { apples: 5, pears: 0, plums: 12 };

console.log(Object.keys(stock));
console.log(Object.values(stock));
console.log(Object.entries(stock));

const inStock = Object.fromEntries(
  Object.entries(stock).filter(([, qty]) => qty > 0).map(([k, qty]) => [k.toUpperCase(), qty]),
);
console.log(inStock);

for (const [fruit, qty] of Object.entries(stock)) console.log(`${fruit.padEnd(7)}${qty}`);
```

Output:

```
[ 'apples', 'pears', 'plums' ]
[ 5, 0, 12 ]
[ [ 'apples', 5 ], [ 'pears', 0 ], [ 'plums', 12 ] ]
{ APPLES: 5, PLUMS: 12 }
apples 5
pears  0
plums  12
```

The pattern **entries → array methods → `fromEntries`** is the idiomatic (natural, conventional for the language) way to "map" or "filter" an object, since objects have no `.map` of their own. All three listing methods return only **own, enumerable, string-keyed** properties — no inherited ones, no symbols — in the key order from section 5.2. (`for...in` also walks inherited enumerable keys, which is why `Object.keys`/`entries` with `for...of` is preferred; see Chapter 3.)

`Object.create` and `Object.is`

```
const animal = { speak() { return `${this.name} makes a sound`; } };
const dog = Object.create(animal);
dog.name = 'Rex';
console.log(dog.speak(), Object.getPrototypeOf(dog) === animal, Object.keys(dog));
const dict = Object.create(null);    // no prototype at all: a "pure" dictionary
console.log('toString' in dict, 'toString' in {});

console.log(NaN === NaN, Object.is(NaN, NaN), 0 === -0, Object.is(0, -0));
```

Output:

```
Rex makes a sound true [ 'name' ]
false true
false true true false
```

- `Object.create(animal)` makes an empty object whose **prototype** — the fallback object consulted when a property is not found — is `animal`. `dog` has only `name` of its own; `speak` is found on the prototype. Prototypes are explained properly in Chapter 9.
- `Object.create(null)` gives an object with *no* prototype — no inherited `toString`, `constructor` or `__proto__` — handy as a dictionary (a plain key-to-value lookup table) whose keys come from users.
- `Object.is` is `===` with two fixes: `NaN` equals itself, and `+0` and `-0` are different. `Map` and `Set` use almost the same rule (they treat `+0` and `-0` as equal).

Property descriptors

Every property has hidden settings called **attributes**, collected in a **property descriptor**:

Attribute	Meaning	Default for normal assignment	Default via `defineProperty`
`value`	The stored value	—	`undefined`
`writable`	Can the value be changed?	`true`	`false`
`enumerable`	Shows up in `Object.keys`, `for...in`, spread, `JSON.stringify`?	`true`	`false`
`configurable`	Can the property be deleted or its attributes changed?	`true`	`false`
`get` / `set`	Accessor functions (instead of `value`/`writable`) — section 5.7	—	`undefined`

From `code/04_descriptors_freeze.mjs`:

```
const tryIt = (label, fn) => {
  try { fn(); console.log(label, '-> ok'); } catch (e) { console.log(label, '->', e.constructor.name); }
};

const product = { name: 'Pen' };
console.log(Object.getOwnPropertyDescriptor(product, 'name'));

Object.defineProperty(product, 'sku', { value: 'P-001' });
console.log(Object.getOwnPropertyDescriptor(product, 'sku'));
console.log(Object.keys(product), Object.getOwnPropertyNames(product), product.sku);
tryIt('write read-only sku', () => { product.sku = 'X'; });
tryIt('delete non-configurable sku', () => { delete product.sku; });

Object.defineProperties(product, {
  price: { value: 2.5, writable: true, enumerable: true, configurable: true },
});
console.log(JSON.stringify(product)); // non-enumerable sku is skipped

const secret = Symbol('secret');
product[secret] = 42;
console.log(Object.keys(product), Object.getOwnPropertySymbols(product));
```

Output:

```
{ value: 'Pen', writable: true, enumerable: true, configurable: true }
{
  value: 'P-001',
  writable: false,
  enumerable: false,
  configurable: false
}
[ 'name' ] [ 'name', 'sku' ] P-001
write read-only sku -> TypeError
delete non-configurable sku -> TypeError
{"name":"Pen","price":2.5}
[ 'name', 'price' ] [ Symbol(secret) ]
```

Notice the asymmetry: a property created by plain assignment has every flag `true`; one created with `defineProperty` has every flag you omit set to `false`. In **strict mode** (modules, classes, `'use strict'`) writing a read-only property throws a `TypeError`; in old **sloppy mode** the write is silently ignored — a nasty source of "why didn't my change stick?" bugs.

Locking an object: preventExtensions, seal, freeze

```
const ext = { a: 1 };  Object.preventExtensions(ext);
const sealed = { a: 1 }; Object.seal(sealed);
const frozen = { a: 1, inner: { b: 1 } }; Object.freeze(frozen);

tryIt('preventExtensions: add   ', () => { ext.z = 1; });
tryIt('preventExtensions: change', () => { ext.a = 2; });
tryIt('preventExtensions: delete', () => { delete ext.a; });
tryIt('seal: add                ', () => { sealed.z = 1; });
tryIt('seal: change             ', () => { sealed.a = 2; });
tryIt('seal: delete             ', () => { delete sealed.a; });
tryIt('freeze: change           ', () => { frozen.a = 2; });
tryIt('freeze: nested change    ', () => { frozen.inner.b = 2; }); // freeze is shallow!
console.log(Object.isExtensible(ext), Object.isSealed(sealed), Object.isFrozen(frozen),
Object.isFrozen(frozen.inner));
```

Output:

```
preventExtensions: add    -> TypeError
preventExtensions: change -> ok
preventExtensions: delete -> ok
seal: add                 -> TypeError
seal: change              -> ok
seal: delete              -> TypeError
freeze: change            -> TypeError
freeze: nested change     -> ok
false true true false
```

Level	Add new keys	Delete keys	Change values	Implemented as
`Object.preventExtensions`	✗	✓	✓	object marked non-extensible
`Object.seal`	✗	✗	✓	+ every property `configurable: false`
`Object.freeze`	✗	✗	✗	+ every data property `writable: false`

All three are **shallow**: `frozen.inner` is still an ordinary, changeable object. A small recursive helper freezes everything:

```
function deepFreeze(obj) {
  for (const value of Object.values(obj)) {
    if (value && typeof value === 'object') deepFreeze(value);
  }
  return Object.freeze(obj);
}
const config = deepFreeze({ db: { host: 'localhost', port: 5432 } });
tryIt('deepFreeze: nested change', () => { config.db.port = 1; });   // -> TypeError
```

Finally, `Object.setPrototypeOf(obj, proto)` changes an existing object's prototype. It works, but engines optimise objects by their shape (the engine's internal record of which properties an object has, in which order), and swapping the prototype later can make code using that object slower — create it with the right prototype via `Object.create` instead.

When not to use it

- Don't `freeze` large hot-path (run very frequently, where speed matters) data "just in case"; it adds little safety in modules you control. Freeze exported constants and configuration, or use TypeScript's `readonly` for compile-time safety (errors caught by a checker before the code runs; TypeScript adds type labels to JavaScript) (Chapter 16).
- Don't reach for `defineProperty` for ordinary data — plain assignment is clearer. Use it for library-style work: hidden (non-enumerable) helpers, read-only constants, lazy getters (which compute a value only the first time it is asked for).

Common mistakes

- Assuming `Object.freeze` is deep.
- Using `defineProperty` and forgetting that omitted flags default to `false` — the property silently becomes hidden and read-only.
- Expecting `Object.keys` to include symbol keys or non-enumerable keys; use `Reflect.ownKeys` for "everything own".

In the interview

30-second answer: "Every property has a descriptor: value, writable, enumerable, configurable — or get/set for accessors. `preventExtensions` stops additions, `seal` also stops deletions, `freeze` also stops writes. All three are shallow, and violations throw only in strict mode."

- *How do you make a truly immutable object?* Deep-freeze recursively, or use immutable data structures (ones that are never changed in place — every update produces a new version; e.g. the Immer library, persistent collections) — `const` alone does nothing for contents.

- *What's the difference between `Object.keys` and `Object.getOwnPropertyNames`?* The latter includes non-enumerable string keys (e.g. an array's `length`).
- *When is `Object.is` different from `===`?* `NaN` vs `NaN` (true) and `0` vs `-0` (false).

Self-check

1. After `Object.seal(o)`, can you do `o.x = 5` for an existing `x`? → Yes, values stay writable.
2. What does `Object.defineProperty(o, 'k', { value: 1 })` make `Object.keys(o)` show? → Not `k` — it is non-enumerable.
3. Which method lists symbol keys? → `Object.getOwnPropertySymbols` (or `Reflect.ownKeys` for all).
4. Is `Object.isFrozen({})` true? → No; but `Object.isFrozen(Object.freeze({}))` is.

5.7 Getters and setters

What it is

An **accessor property** looks like a normal property from the outside but runs a function when read (**getter**, `get`) or written (**setter**, `set`). It is how you compute a value on demand or validate an assignment without changing how callers use the object.

How it works

From `code/05_getters_setters_optional.mjs`:

```
const account = {
  owner: 'Sam',
  _balance: 100,                    // underscore = "please treat as internal" (convention only)
  get balance() { return `$${this._balance.toFixed(2)}`; },
  set balance(v) {
    if (typeof v !== 'number' || v < 0) throw new RangeError('balance must be a non-negative number');
    this._balance = v;
  },
  deposit(amount) { this._balance += amount; return this; }, // returning this enables chaining
};

console.log(account.balance);           // getter runs; no parentheses
account.balance = 250;                  // setter runs
console.log(account.deposit(10).deposit(5).balance);
try { account.balance = -1; } catch (e) { console.log(e.name + ':', e.message); }
console.log(Object.getOwnPropertyDescriptor(account, 'balance'));

const rect = { w: 3, h: 4, get area() { return this.w * this.h; } };
rect.w = 10;
console.log(rect.area);
```

Output:

```
$100.00
$265.00
RangeError: balance must be a non-negative number
{
  get: [Function: get balance],
  set: [Function: set balance],
  enumerable: true,
  configurable: true
}
40
```

1. `account.balance` (no parentheses) calls the getter.
2. `account.balance = 250` calls the setter with `250`; the setter validates and stores into the "backing" property `_balance`.
3. The descriptor has `get`/`set` instead of `value`/`writable`.

4. `rect.area` is **derived** — always up to date with `w` and `h`, never stored.

The getter and the backing property **must have different names**; a setter that does `this.balance = v` would call itself forever. The underscore is only a convention; truly private state uses closures (Chapter 4) or class `#private` fields (Chapter 9).

When not to use it

Getters should be cheap and side-effect free — readers assume a property read is instant. Anything slow (network, heavy computation) belongs in an explicit method like `loadBalance()`. A getter with no setter makes the property read-only (the write throws in strict mode).

Real-world examples

Vue's reactivity (v2; Vue is a UI framework, and *reactivity* means the screen updates automatically when data changes) was built on `defineProperty` getters/setters; MobX and many ORMs (Object-Relational Mappers — libraries that map database rows to objects; e.g. computed fields in models) use accessors; DOM properties such as `element.innerHTML` and `input.value` are accessors on prototypes.

5.8 Optional chaining `?.`

The problem it solves

Data from APIs is often incomplete. Reading `res.data.user.name` throws `TypeError: Cannot read properties of undefined` if any link in the chain is missing. Before ES2020 people wrote `res && res.data && res.data.user && res.data.user.name`.

How it works

`a?.b` means: "if `a` is `null` or `undefined`, stop and return `undefined`; otherwise read `a.b`". It comes in three forms: `obj?.prop`, `obj?.[expr]`, and `fn?.(args)`. The whole rest of the chain is skipped once a `?.` short-circuits.

```
const res = { data: { user: { name: 'Ana', tags: ['a'] } } };
const empty = {};
console.log(res.data?.user?.name, empty.data?.user?.name);
console.log(res.data?.user?.tags?.[0], empty.data?.user?.tags?.[0]);   // with brackets
console.log(res.logout?.(), typeof empty.save?.());                    // calling a maybe-missing method
console.log(empty.data?.user?.name ?? 'Guest');                        // pair with ?? for a default
```

Output:

```
Ana undefined
a undefined
undefined undefined
Guest
```

Pair `?.` with the **nullish coalescing operator** `??` (Chapter 2) to supply a default only when the result is `null`/`undefined` (unlike `||`, which also replaces `0`, `''` and `false`).

Common mistakes

- Sprinkling `?.` everywhere. If a value *must* exist, let the code fail loudly — hiding a bug behind `undefined` makes it surface far away.
- `?.` cannot be used on the left side of an assignment: `obj?.x = 1` is a `SyntaxError`.
- `?.` only guards against `null`/`undefined`; `(5)?.toFixed` still works and `''?.length` is `0`.

5.9 JavaScript's built-in (native) objects

JavaScript ships with a standard library of **built-in objects** available in every environment (browser, Node, Deno, Bun) with no import. Some are **constructors** (you call them with `new`, or as converters), some are **namespaces** (plain objects holding static functions, never instantiated):

Built-in	Kind	Used for	Where in this book
`Object`	constructor + static helpers	Base of all objects; `Object.keys`, etc.	this chapter
`String`, `Number`, `Boolean`, `BigInt`, `Symbol`	wrappers / converters	Methods on primitives; conversions	Ch 2, Ch 6
`Array`	constructor	Ordered lists	Ch 7
`Function`	constructor	Every function is an object of this type	Ch 4
`Date`	constructor	Points in time	5.11
`RegExp`	constructor	Pattern matching on text	Ch 6
`Map`, `Set`, `WeakMap`, `WeakSet`, `WeakRef`	constructors	Keyed collections, unique values	Ch 8
`Promise`	constructor	Results that arrive later	Ch 12
`Error` (+ `TypeError`, `RangeError`, …)	constructors	Exceptions (error objects that are thrown to stop normal flow)	Ch 11
`Math`	namespace	Mathematical constants and functions	5.10
`JSON`	namespace	Converting to/from JSON text	5.12
`Intl`	namespace	Locale-aware (adapting to a language and region, e.g. `en-US`) formatting (dates, numbers, lists)	5.11, Ch 6
`Reflect`, `Proxy`	namespace / constructor	Meta-programming: `Reflect` exposes object operations as functions (`Reflect.ownKeys`, 5.2); a `Proxy` wraps an object and intercepts reads and writes (how Vue 3 reactivity and Immer work)	overview only
`ArrayBuffer`, typed arrays	constructors	Raw binary data	Ch 7
`globalThis`	the global object	Access to globals in any environment	Ch 1

Environment objects such as `document`, `window`, `fetch` or Node's `process` are **not** part of the language; they come from the host (the program that runs the engine: browser or Node) — see Chapters 13–15.

5.10 `Math`

What it is

`Math` is a **namespace object**: a plain object holding mathematical constants and functions as properties. It is not a constructor — you never write `new Math()` (that throws a `TypeError`); you call `Math.floor(x)` directly. In class terminology all its members are **static** (belong to `Math` itself rather than to instances).

Constants

Property	Value (approx.)	Meaning
`Math.PI`	3.14159	Circle circumference ÷ diameter
`Math.E`	2.71828	Euler's number, base of natural logarithms
`Math.SQRT2` / `Math.SQRT1_2`	1.41421 / 0.70711	√2 and √½
`Math.LN2` / `Math.LN10`	0.69315 / 2.30259	Natural log of 2 and of 10
`Math.LOG2E` / `Math.LOG10E`	1.44270 / 0.43429	$\log_2(e)$ and $\log_{10}(e)$

Methods by family

Family	Methods	Notes
Rounding	`round`, `floor`, `ceil`, `trunc`, `fround`	See the table below for negatives
Sign & size	`abs`, `sign`	`sign` returns -1, 0, 1 (or -0/NaN)
Extremes	`max(...xs)`, `min(...xs)`	Take separate arguments — spread arrays in
Powers & roots	`pow(x, y)` (same as `x ** y`), `sqrt`, `cbrt`, `hypot(...xs)` (√ of sum of squares)	
Exp & logs	`exp`, `expm1`, `log` (natural), `log10`, `log2`, `log1p`	`log(0)` is `-Infinity`; negative → `NaN`
Trigonometry (radians — an angle unit where a full circle is 2π)	`sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `atan2(y, x)`, and hyperbolic `sinh`, `cosh`, `tanh`, …	`atan2` gives the angle of point (x, y), full −π…π range
Random	`random()`	Float in **`[0, 1)`** — 0 possible, 1 never
Integer tricks	`clz32`, `imul`	Low-level 32-bit helpers

Newer additions: `Math.f16round` (ES2025, rounds to half-precision float; in Node 24) and `Math.sumPrecise` (ES2026, adds a list of numbers without accumulated rounding error — not yet in Node 24, so check support before use).

How it works — examples

From `code/06_math.mjs`:

```
const r = 5;
console.log('area', Math.PI * r ** 2, Math.PI * Math.pow(r, 2));

const rows = [3.56, 3.5, -3.5, -3.56, 2.4].map((x) => ({
  x, round: Math.round(x), floor: Math.floor(x), ceil: Math.ceil(x), trunc: Math.trunc(x),
}));
console.table(rows);

console.log(Math.round(1.2345 * 100) / 100, (1.005).toFixed(2), (1.2345).toFixed(2));

const temps = [18, 25, 21, 30, 17];
console.log(Math.max(10, 20, 30, 40, 50), Math.min(...temps), Math.max(...temps));
console.log(Math.max(), Math.min(), Math.max(1, 'x'));

const clamp = (v, lo, hi) => Math.min(Math.max(v, lo), hi);
console.log(clamp(120, 0, 100), clamp(-5, 0, 100), clamp(42, 0, 100));
```

Output:

```
area 78.53981633974483 78.53981633974483
```

(index)	x	round	floor	ceil	trunc
0	3.56	4	3	4	3
1	3.5	4	3	4	3
2	-3.5	-3	-4	-3	-3
3	-3.56	-4	-4	-3	-3
4	2.4	2	2	3	2

```
1.23 1.00 1.23
50 17 30
-Infinity Infinity NaN
100 0 42
```

What to notice:

- **`round` rounds halves up toward +∞**: `Math.round(-3.5)` is `-3`, not `-4`. This is different from "round half away from zero" in many other languages.
- **`floor` vs `trunc`** only differ for negatives: floor goes down (−3.5 → −4), trunc chops the fraction (−3.5 → −3).
- **`toFixed` returns a string** and suffers from binary floating point: `1.005` is stored as 1.00499999…, so `(1.005).toFixed(2)` is `"1.00"`. For money, work in integer cents or use `Intl.NumberFormat` for display (see Chapter 2 on float precision).
- `Math.max()` with no arguments is `-Infinity` (the identity for "max" — a starting value that never changes the answer), and any argument that converts to `NaN` (such as `'x'`) makes the result `NaN` — numeric strings like `'5'` are converted and work. Spreading a *huge* array into `max`/`min` can exceed the engine's argument limit; use `reduce` for arrays with hundreds of thousands of items.

```
console.log(Math.abs(-7), Math.sign(-7), Math.sqrt(81), Math.cbrt(27), Math.hypot(3, 4));
console.log(Math.pow(2, 10), 2 ** 10, Math.exp(1), Math.log(Math.E), Math.log10(1000), Math.log2(8));

const toRad = (deg) => (deg * Math.PI) / 180;
console.log(Math.sin(toRad(30)).toFixed(4), Math.cos(toRad(60)).toFixed(4), Math.sin(Math.PI));
console.log('angle of (1,1):', (Math.atan2(1, 1) * 180) / Math.PI, 'deg');
```

Output:

```
7 -1 9 3 5
1024 1024 2.718281828459045 1 3 3
0.5000 0.5000 1.2246467991473532e-16
angle of (1,1): 45 deg
```

Trigonometric functions take **radians** (a full circle is 2π radians = 360°). `Math.sin(Math.PI)` is not exactly 0 because π itself cannot be stored exactly.

Random integers

`Math.random()` returns a float `x` with `0 ≤ x < 1`. To get an integer between `min` and `max` **inclusive**:

1. Multiply by the number of possible results, `max - min + 1` → float in `[0, count)`.
2. `Math.floor` → integer `0 … count-1`.
3. Add `min` → integer `min … max`.

```
const randInt = (min, max) => Math.floor(Math.random() * (max - min + 1)) + min;
```

The demo draws 30,000 values from `randInt(1, 3)` and confirms all land in `{1, 2, 3}` roughly evenly. Using `Math.round` instead of `floor` is a classic bug: the two end values then get only half the probability of the middle ones.

`Math.random` is not cryptographically secure (unpredictable enough to resist attackers) — its output can be predicted. For tokens, IDs, passwords or anything security-related use the Web Crypto API: `crypto.randomUUID()` (a UUID is a random 128-bit identifier, practically guaranteed to be unique) or `crypto.getRandomValues(new Uint32Array(1))` (global in browsers and Node).

In the interview

- *Why does `0.1 + 0.2 !== 0.3`, and how do you round to two decimals?* IEEE-754 binary floats (the standard binary format computers use for decimal numbers) can't store 0.1 exactly. Compare with a tolerance (`Math.abs(a - b) < Number.EPSILON`), round with `Math.round(x * 100) / 100`, or use integers (cents).
- *Random integer in [a, b]?* `Math.floor(Math.random() * (b - a + 1)) + a`.
- *Max of an array?* `Math.max(...arr)` for normal sizes; `arr.reduce((m, x) => x > m ? x : m)` for very large ones.

Self-check

1. `Math.round(-2.5)`? → `-2`.
2. `Math.trunc(-4.7)` vs `Math.floor(-4.7)`? → `-4` vs `-5`.
3. Can `Math.random()` return 1? → No; `[0, 1)`.
4. Convert 180° to radians. → `Math.PI`.

5.11 `Date`

What it is

A `Date` object represents **one instant in time**, stored internally as a single number: the milliseconds elapsed since the **Unix epoch**, 1 January 1970 00:00:00 **UTC** (Coordinated Universal Time — the time-zone-free reference clock). Everything else — year, month, hour, weekday — is *computed* from that number, either in **local time** (the time zone of the computer running the code) or in UTC.

That single fact explains most `Date` surprises: a `Date` has no time zone of its own; the time zone comes in only when you read or print it.

Why it matters

Calendars, reminders, booking systems, logs, subscriptions, "posted 3 hours ago" — all need time. `Date` is also infamous for pitfalls (0-based months, mutability, parsing quirks, DST — daylight-saving time, when clocks jump an hour in spring and autumn), which is why the language now has **Temporal** (end of this section).

Creating dates

Form	Creates	Example
`new Date()`	The current instant	`new Date()`
`new Date(ms)`	Instant `ms` milliseconds after the epoch (negative = before 1970)	`new Date(0)` → 1970-01-01T00:00:00Z
`new Date(string)`	Parsed from text; reliable only for **ISO 8601** (the international standard date-time text format: `YYYY-MM-DDTHH:mm:ss.sssZ`)	`new Date('2024-03-10T14:30:00Z')`
`new Date(y, m, d?, h?, min?, s?, ms?)`	Built from **local-time** parts; **month is 0-based** (0 = January, 11 = December); omitted day = 1, omitted time parts = 0	`new Date(2024, 0, 31, 9, 15)`
`Date.now()`	Not a Date — just the current ms number	handy for timing
`Date.UTC(y, m, …)`	ms number for those parts **in UTC**	`new Date(Date.UTC(2024, 0, 1))`

Calling `Date()` *without* `new` returns a string, not a Date — always use `new`.

From `code/07_date.mjs` (the script pins the time zone with `process.env.TZ = 'America/New_York'` — `process.env` holds environment variables, settings passed to a program from outside — so output is

repeatable):

```
const now = new Date();                                  // current moment
console.log('now is a Date:', now instanceof Date, typeof now.getTime());
const epoch = new Date(0);                               // ms since 1970-01-01T00:00:00Z
const fromMs = new Date(1_700_000_000_000);
const fromIso = new Date('2024-03-10T14:30:00Z');        // ISO 8601 string (Z = UTC)
const fromParts = new Date(2024, 0, 31, 9, 15);          // local time; month is 0-based (0 = Jan)
console.log(epoch.toISOString(), fromMs.toISOString());
console.log(fromIso.toISOString(), fromParts.toString());
```

Output:

```
now is a Date: true number
1970-01-01T00:00:00.000Z 2023-11-14T22:13:20.000Z
2024-03-10T14:30:00.000Z Wed Jan 31 2024 09:15:00 GMT-0500 (Eastern Standard Time)
```

Parsing pitfalls

```
console.log(new Date('2023-01-01').toString());
console.log(new Date('2023-01-01T00:00').toString());
console.log(new Date('not a date').getTime(), String(new Date('nope')));
```

Output:

```
Sat Dec 31 2022 19:00:00 GMT-0500 (Eastern Standard Time)
Sun Jan 01 2023 00:00:00 GMT-0500 (Eastern Standard Time)
NaN Invalid Date
```

- A **date-only** ISO string (`'2023-01-01'`) is treated as **UTC midnight** — which in New York is still 31 December. A date-*time* string without `Z` or an offset is treated as **local**. This inconsistency is in the spec, for historical reasons.
- Non-ISO strings (`'March 25, 2015'`, `'03/25/2015'`) are parsed in an implementation-specific way (each engine may do it differently). They usually work in modern engines, but the spec does not guarantee them — prefer ISO strings or numeric parts.
- An unparseable string gives an **Invalid Date**: a Date whose time value is `NaN`. Test with `Number.isNaN(d.getTime())`.

Reading components — getters

Local getter	UTC twin	Returns
`getFullYear()`	`getUTCFullYear()`	4-digit year (never use the obsolete `getYear`)
`getMonth()`	`getUTCMonth()`	**0–11**
`getDate()`	`getUTCDate()`	Day of month **1–31**
`getDay()`	`getUTCDay()`	Day of week **0–6**, 0 = Sunday
`getHours()` / `getMinutes()` / `getSeconds()` / `getMilliseconds()`	`getUTC…`	0–23 / 0–59 / 0–59 / 0–999
`getTime()` / `valueOf()`	—	ms since the epoch (same instant in any zone)
`getTimezoneOffset()`	—	Minutes to add to local time to get UTC (positive west of Greenwich, e.g. 240 for New York in summer)

```
const d = new Date('2024-07-04T23:30:00Z');
console.log('local:', d.getFullYear(), d.getMonth(), d.getDate(), d.getDay(), d.getHours(), d.getMinutes());
console.log('UTC  :', d.getUTCFullYear(), d.getUTCMonth(), d.getUTCDate(), d.getUTCDay(), d.getUTCHours());
console.log('offset minutes (UTC - local):', d.getTimezoneOffset());
```

Output:

```
local: 2024 6 4 4 19 30
UTC  : 2024 6 4 4 23
offset minutes (UTC - local): 240
```

Month `6` is **July**; weekday `4` is **Thursday**. The same instant is 19:30 in New York and 23:30 UTC.

Changing components — setters

Setters (`setFullYear`, `setMonth`, `setDate`, `setHours`, `setMinutes`, `setSeconds`, `setMilliseconds`, `setTime`, plus `setUTC…` twins) **mutate** the Date in place. Two properties make them useful:

1. Some accept several parts at once: `setFullYear(year, month?, day?)`, `setHours(h, m?, s?, ms?)`.
2. Out-of-range values **roll over** into the next unit — day 32 of January becomes 1 February, day 0 becomes the last day of the previous month.

```
const b = new Date(2024, 5, 15);
b.setFullYear(1990, 0, 1);                  // year, month, day in one call
console.log(b.toDateString());
const roll = new Date(2024, 0, 31);
roll.setMonth(1);                           // Feb 31 does not exist -> Mar 2 (2024 is a leap year)
console.log(roll.toDateString());
const addDays = (date, n) => { const c = new Date(date); c.setDate(c.getDate() + n); return c; };
console.log(addDays(new Date(2024, 11, 30), 5).toDateString());
const lastDayOfFeb = new Date(2024, 2, 0).getDate();   // day 0 of March = last day of Feb
console.log('days in Feb 2024:', lastDayOfFeb);
```

Output:

```
Mon Jan 01 1990
Sat Mar 02 2024
Sat Jan 04 2025
days in Feb 2024: 29
```

`addDays` copies first (`new Date(date)`) so the caller's Date isn't mutated, then relies on rollover to cross the year boundary. The "month-end overflow" (`Jan 31 + 1 month = Mar 2`) is a real bug source in billing code — decide explicitly whether you want the last day of the month instead.

Differences and comparisons

Subtracting two Dates converts both to milliseconds (via `valueOf`) and gives the difference as a number.

```
const start = new Date('2023-01-01');
const end = new Date('2023-12-31');
const MS_PER_DAY = 1000 * 60 * 60 * 24;
console.log((end - start) / MS_PER_DAY + ' days');
const beforeDst = new Date(2024, 2, 9), afterDst = new Date(2024, 2, 11);
console.log((afterDst - beforeDst) / MS_PER_DAY, Math.round((afterDst - beforeDst) / MS_PER_DAY));

const x = new Date(2024, 0, 1), y = new Date(2024, 0, 1);
console.log(x === y, x.getTime() === y.getTime(), x < addDays(y, 1));
```

Output:

```
364 days
1.9583333333333333 2
false true true
```

- 1 Jan to 31 Dec is 364 days apart (add 1 if you count both ends).
- Across the March **daylight-saving time** (DST) switch, two local midnights are only 47 hours apart, giving 1.958 "days". Round, or compute in UTC.
- `<` and `>` work on Dates (they compare the numbers), but `===` compares object identity. Compare `getTime()` values for equality.

Formatting

Method	Output (for the instant 2024-03-10 14:30 UTC, local zone New York)	Use for
`toISOString()`	`2024-03-10T14:30:00.000Z`	Storage, APIs, logs — always UTC, sortable
`toJSON()`	same as ISO	Used automatically by `JSON.stringify`
`toString()`	`Sun Mar 10 2024 10:30:00 GMT-0400 (Eastern Daylight Time)`	Debugging
`toDateString()` / `toTimeString()`	`Sun Mar 10 2024` / `10:30:00 GMT-0400 (…)`	Debugging
`toUTCString()`	`Sun, 10 Mar 2024 14:30:00 GMT`	HTTP headers (extra name/value lines sent with a web request or response)
`toLocaleDateString(locale, opts)` / `toLocaleTimeString` / `toLocaleString`	`3/10/2024` (en-US), `10/03/2024` (en-GB), `10.3.2024` (de-DE)	Showing dates to people
`Intl.DateTimeFormat(locale, opts).format(d)`	reusable formatter	Formatting many dates
`Intl.RelativeTimeFormat`	`yesterday`, `in 3 weeks`	"time ago" labels

```
console.log(t.toLocaleString('en-US', { timeZone: 'Asia/Kolkata', dateStyle: 'full', timeStyle: 'short' }));
const fmt = new Intl.DateTimeFormat('en', { month: 'short', day: 'numeric', year: 'numeric', timeZone: 'UTC' });
console.log(fmt.format(t));
const rtf = new Intl.RelativeTimeFormat('en', { numeric: 'auto' });
console.log(rtf.format(-1, 'day'), '|', rtf.format(3, 'week'));
```

Output:

```
Sunday, March 10, 2024 at 8:00 PM
Mar 10, 2024
yesterday | in 3 weeks
```

The `timeZone` option lets you *display* an instant in any IANA zone (the official worldwide time-zone database names: `'Asia/Kolkata'`, `'Europe/Berlin'`) — but `Date` itself still cannot *do arithmetic* in a chosen zone. That gap is what Temporal fills.

Temporal — the modern replacement (status in 2026)

Temporal is a new global namespace of immutable date/time types designed to fix `Date`'s problems. Verified status (September 2026):

- It reached **Stage 4** (finished) at the TC39 meeting of **March 2026**; TC39's finished-proposals list slots it into the **ES2027** edition (some articles say "ES2026"; the ES2026 snapshot was already cut).
- It is shipped by default in **Firefox 139+**, **Chrome/Edge 144+** and **Node.js 26+**. **Safari** stable did not yet ship it at the time of writing, and **Node 24** (this book's runtime) does not have it — our script prints `Temporal available here: false`. Use the `@js-temporal/polyfill` or `temporal-polyfill` package where it's missing (a **polyfill** is code that adds a missing built-in feature to older environments).

Its main types:

Type	Represents	Example
`Temporal.Instant`	An exact moment (like a Date's number, in nanoseconds)	a log timestamp
`Temporal.ZonedDateTime`	Instant + time zone + calendar; DST-aware arithmetic	a meeting in Berlin
`Temporal.PlainDate` / `PlainTime` / `PlainDateTime`	Wall-clock values with **no** time zone	a birthday; a store's opening time
`Temporal.PlainYearMonth` / `PlainMonthDay`	Partial dates	a credit-card expiry; an anniversary
`Temporal.Duration`	A length of time	"2 hours 30 minutes"
`Temporal.Now`	Current time in various forms	`Temporal.Now.plainDateISO()`

A taste (Node 26 / Chrome 144 / Firefox 139 — not run in this book's Node 24):

```
const due = Temporal.PlainDate.from('2024-01-31').add({ months: 1 });
due.toString();                                  // '2024-02-29' — clamps to month end, months are 1-based
const meeting = Temporal.ZonedDateTime.from('2026-03-28T09:00[Europe/Berlin]');
meeting.add({ days: 1 }).toString();             // DST-aware: still 09:00 local the next day
```

Why it's better: values are **immutable** (every operation returns a new object), months are **1-based**, parsing is strict ISO, time zones and calendars are first-class, and there are separate types for "an instant" and "a date on a wall calendar". Until Temporal is everywhere you target, libraries like **date-fns**, **Luxon** or **Day.js** fill the gap.

Common mistakes

- Forgetting that months are 0-based (`new Date(2024, 1, 1)` is **February** 1).
- Confusing `getDate()` (day of month) with `getDay()` (day of week).
- Parsing `'2023-01-01'` and being surprised it shows 31 December in the Americas.
- Mutating a Date that other code also holds (setters change it in place). Copy with `new Date(d)` first.
- Storing local-time strings in databases. Store UTC (ISO strings or epoch ms) and convert only for display.
- Computing "days between" with raw division across DST.

Real-world examples

- APIs send ISO 8601 strings (`"2024-05-01T08:00:00Z"`); clients parse with `new Date(str)` and display with `Intl.DateTimeFormat`.
- Performance timing uses `Date.now()` (or better, `performance.now()` which is monotonic — it only ever goes forward, even if the system clock is changed — and sub-millisecond).
- JWT tokens (JSON Web Tokens — signed login tokens passed between client and server) store expiry as **seconds** since the epoch — multiply by 1000 before passing to `new Date`.

In the interview

30-second answer: "A Date is a wrapper around one number — milliseconds since 1970 UTC. Getters compute local or UTC parts from it. Months are 0-based, setters mutate and roll over, date-only ISO strings parse as UTC. For display I use `Intl.DateTimeFormat`; for serious date math I use Temporal (now standard, shipping in Chrome, Firefox and Node 26) or a library."

- *How do you get the number of days in a month?* `new Date(year, month + 1, 0).getDate()` — day 0 of the next month.
- *How do you check a Date is valid?* `!Number.isNaN(d.getTime())`.

- *Why might* `new Date('2024-01-01')` *show the wrong day?* It's parsed as UTC midnight, then shown in local time west of UTC.

Self-check

1. `new Date(2025, 11, 25).getMonth()`? → `11` (December).
2. What does `getDay()` return for a Sunday? → `0`.
3. `new Date(2024, 0, 32)` is which date? → 1 February 2024.
4. How many ms in a day? → 86,400,000 (ignoring DST days).
5. Which format should you store? → UTC — ISO 8601 or epoch milliseconds.

5.12 JSON

What it is

JSON (JavaScript Object Notation) is a *text* format for structured data, standardised as ECMA-404 / RFC 8259 (an RFC is a numbered internet standards document). It borrows JavaScript's literal syntax but is much stricter, and it is language-independent — Python, Java, Go and every database speak it. The global `JSON` namespace has two functions:

- `JSON.stringify(value, replacer?, space?)` → turns a value into a JSON **string** (serialisation).
- `JSON.parse(text, reviver?)` → turns a JSON string back into values (deserialisation).

JSON syntax rules (vs JS literals)

Allowed in JSON	Not allowed in JSON (but fine in JS)
Objects `{}` with **double-quoted** keys	Unquoted or single-quoted keys: `{a: 1}`, `{'a': 1}`
Strings in **double quotes**	Single quotes, template literals
Numbers (decimal), `true`, `false`, `null`	`undefined`, `NaN`, `Infinity`, hex `0xFF`, BigInt
Arrays `[]`	Trailing commas `[1, 2,]`
Nesting to any depth	Comments, functions, Dates, Maps, Sets, symbols

How `stringify` works

From `code/08_json.mjs`:

```
const order = {
  id: 42, customer: 'Ravi', total: 19.99, paid: true, coupon: null,
  items: [{ sku: 'A1', qty: 2 }],
  placedAt: new Date('2024-05-01T08:00:00Z'),
};

const text = JSON.stringify(order);
console.log(text);
console.log(typeof text, text.length);
console.log(JSON.stringify({ a: 1, b: [1, 2] }, null, 2));
console.log(JSON.stringify(order, ['id', 'total']));
console.log(JSON.stringify(order, (key, value) => (key === 'customer' ? undefined : value)).slice(0, 60) + '...');
```

Output:

```
{"id":42,"customer":"Ravi","total":19.99,"paid":true,"coupon":null,"items":
[{"sku":"A1","qty":2}],"placedAt":"2024-05-01T08:00:00.000Z"}
string 136
{
  "a": 1,
  "b": [
    1,
    2
  ]
}
{"id":42,"total":19.99}
{"id":42,"total":19.99,"paid":true,"coupon":null,"items":[{"...
```

1. The result is a **string**. The Date became an ISO string because `Date.prototype.toJSON` exists.
2. The third argument `space` (a number of spaces up to 10, or a string such as `'\t'`) pretty-prints.
3. The second argument, the **replacer**, is either an **array** of allowed keys, or a **function** `(key, value) => newValue` called for every property (return `undefined` to drop it).

What JSON silently changes or refuses

```
const weird = {
  f() {}, u: undefined, s: Symbol('x'), nan: NaN, inf: Infinity, negZero: -0,
  map: new Map([['k', 1]]), set: new Set([1]), arr: [undefined, () => 1, NaN],
};
console.log(JSON.stringify(weird));
try { JSON.stringify({ big: 10n }); } catch (e) { console.log('BigInt ->', e.constructor.name); }
const loop = { name: 'me' }; loop.self = loop;
try { JSON.stringify(loop); } catch (e) { console.log('circular ->', e.constructor.name); }

const money = { cents: 1999, toJSON() { return (this.cents / 100).toFixed(2); } };
console.log(JSON.stringify({ price: money }));
```

Output:

```
{"nan":null,"inf":null,"negZero":0,"map":{},"set":{},"arr":[null,null,null]}
BigInt -> TypeError
circular -> TypeError
{"price":"19.99"}
```

Value	In an object property	In an array	At top level
`undefined`, function, symbol	**property dropped**	becomes `null`	`JSON.stringify` returns `undefined`
`NaN`, `Infinity`	`null`	`null`	`"null"`
`-0`	`0`	`0`	`"0"`
`Date`	ISO string (via `toJSON`)	same	same
`Map`, `Set`	`{}` — data lost!	same	same
BigInt	**throws** `TypeError`	throws	throws
Circular reference	**throws** `TypeError`	throws	—
Non-enumerable or symbol keys	skipped	—	—

Any object can define **`toJSON()`** to control its own representation — here a money object serialises as a decimal string. To send a `Map`, convert it first: `Object.fromEntries(map)` or `[...map]`.

How `parse` works

```
const parsed = JSON.parse(text);
console.log(parsed.items[0].qty, typeof parsed.placedAt);          // dates come back as strings
const revived = JSON.parse(text, (key, value) => (key === 'placedAt' ? new Date(value) : value));
console.log(revived.placedAt instanceof Date, revived.placedAt.getUTCHours());

for (const bad of ["{'a': 1}", '{a: 1}', '{"a": 1,}', 'undefined', '']) {
  try { JSON.parse(bad); console.log('ok', bad); } catch (e) { console.log('SyntaxError for',
JSON.stringify(bad)); }
}
console.log(JSON.parse('"just a string"'), JSON.parse('3.5'), JSON.parse('[true,null]'));
console.log(JSON.parse('{"id": 9007199254740993}').id);
```

Output:

```
2 string
true 8
SyntaxError for "{'a': 1}"
SyntaxError for "{a: 1}"
SyntaxError for "{\"a\": 1,}"
SyntaxError for "undefined"
SyntaxError for ""
just a string 3.5 [ true, null ]
9007199254740992
```

- JSON has no date type: dates come back as **strings**. A **reviver** function `(key, value) => newValue` runs on every value (innermost first) and can convert them back.
- Invalid JSON throws a `SyntaxError` — always wrap `JSON.parse` of external input in `try/catch`.
- Any JSON value can be the top level, not only objects.
- Integers above `Number.MAX_SAFE_INTEGER` ($2^{53} - 1$) lose precision: `9007199254740993` became `…992`. APIs such as Twitter/X send large IDs as strings for exactly this reason. The ES2026 feature **"JSON.parse source text access"** gives the reviver a third argument whose `source` property is the original text, so you can rebuild an exact BigInt (Node 24 already supports it):

```
const exact = JSON.parse('{"id": 9007199254740993}', (key, value, ctx) =>
  key === 'id' ? BigInt(ctx.source) : value);
console.log(exact.id);   // 9007199254740993n
```

Deep copy via JSON vs `structuredClone`

```
const src = { when: new Date(0), tags: new Set(['x']), n: 1 };
console.log(JSON.parse(JSON.stringify(src)), structuredClone(src));
```

Output:

```
{ when: '1970-01-01T00:00:00.000Z', tags: {}, n: 1 } { when: 1970-01-01T00:00:00.000Z, tags: Set(1) { 'x' }, n: 1
}
```

The JSON round-trip turned the Date into a string and emptied the Set; `structuredClone` kept both. Use `structuredClone` for copying; use JSON for **transport** (HTTP bodies, files, `localStorage` (the browser's small built-in key-value store), message queues (services that hold messages until another program picks them up — see System Design Fundamentals)).

Real-world examples

- `fetch` responses: `await response.json()` is `JSON.parse` on the body; sending uses `body: JSON.stringify(data)` with `Content-Type: application/json` (Chapter 14).
- `package.json` (a project's settings and dependency list), `tsconfig.json` (TypeScript's settings) and most config files are JSON (tsconfig actually allows comments — it is "JSONC", not strict JSON).
- `localStorage` stores only strings, so objects go through `JSON.stringify` / `JSON.parse`.

- Logging pipelines emit one JSON object per line ("NDJSON") so tools like Elasticsearch (a search engine often used to store and search logs) can index fields (build fast lookup structures for them).

In the interview

> **30-second answer:** "`JSON.stringify` serialises to a string, dropping functions/undefined/symbols in objects, turning NaN/Infinity into null, Dates into ISO strings via `toJSON`, and throwing on BigInt and cycles. `JSON.parse` is strict — double quotes, no trailing commas — and takes a reviver to restore types like Dates."

- *How do you handle circular structures?* Use a replacer with a `WeakSet` (a set of objects that doesn't keep them alive in memory, Chapter 8) of seen objects to drop or mark repeats, or restructure the data; `structuredClone` can *copy* cycles but that doesn't help sending JSON.
- *Why does `JSON.stringify({a: undefined})` give `{}`?* `undefined` properties are omitted.
- *Is JSON a subset of JavaScript?* Since ES2019 ("JSON superset"), yes — every valid JSON text is a valid JS expression.

Self-check

1. `JSON.stringify([undefined])`? → `"[null]"`.
2. `JSON.stringify({ d: new Date(0) })`? → `{"d":"1970-01-01T00:00:00.000Z"}`.
3. Is `{"a":1,}` valid JSON? → No — trailing comma.
4. What's the reviver's job? → Transform parsed values (e.g., strings back into Dates).

Chapter TL;DR

- **Objects** are key → value collections; create with literals `{}`; use **dot** access for fixed identifier keys and **bracket** access for dynamic or unusual keys. Computed keys `[expr]` and shorthand `{ name }` shorten literals.
- Key order is defined: integer-like keys ascending, then string keys by insertion, then symbols.
- Variables hold **references**; `===` compares identity; `const` doesn't freeze contents.
- **Spread** `{...a, ...b}` and **`Object.assign`** are **shallow**; later keys win. **`structuredClone`** deep-copies.
- **Destructuring** unpacks properties with renames (`{ id: orderId }`), defaults (only for `undefined`), nesting, and in parameters.
- **`Object.keys/values/entries/fromEntries`** list and rebuild objects; `hasOwn` checks own keys; `Object.is` fixes `NaN` and `-0`; `Object.create(proto | null)` sets the prototype.
- **Descriptors** (`writable`, `enumerable`, `configurable`, `get`/`set`) control behaviour; `preventExtensions` < `seal` < `freeze`, all shallow.
- **Getters/setters** compute or validate on property access; keep them cheap.
- **`?.`** short-circuits on `null`/`undefined`; pair with **`??`** for defaults.
- **`Math`** is a namespace: `round` (halves toward +∞), `floor`/`ceil`/`trunc`, `max/min(...arr)`, `**`, radians for trig; random int = `Math.floor(Math.random() * (max - min + 1)) + min`; use `crypto` for security.
- **`Date`** = ms since 1970 UTC; months 0-based, `getDay` is weekday, setters mutate and roll over, date-only ISO parses as UTC; format with `Intl`. **Temporal** is standard (Stage 4, March 2026) and shipping in Chrome, Firefox and Node 26, not Node 24/Safari.
- **`JSON.stringify`** drops/changes non-JSON values and throws on BigInt/cycles; **`JSON.parse`** is strict and takes a reviver.